

Chapters 2 and 3 — Working Notes

Bullets to turn into paragraphs. Every number computed live from `data/processed/nodes.json`.

How to use this with Nadine. One of you reads a bullet block aloud, the other types what you say about it. Do not edit while drafting — finish the subsection first. The numbers are already verified, so you never have to stop and look anything up.

CHAPTER 2 — Data Understanding

2.0 Chapter introduction

- this is the CRISP-DM data understanding phase — examine the source before building anything
- why it gets a whole chapter: the extraction rules in Chapter 3 are all derived from findings here
- two halves: what the document contains (2.1–2.2), and what makes it hard to extract (2.3)

Land: *You cannot design an extraction rule until you know what the page is actually doing.*

2.1.1 Source and Context

- AfDB Delegation of Authority Matrix, internal governance document
- over 250 pages in total; **79-page subset** taken as scope
- within that subset, activity content sits on **pages 14–78**, and **37 distinct pages** actually carry activity tables
- **three chapters** covered — chapter 3 is Non-Sovereign Operations (NSO)
- say why the subset: scope decision made with the supervisors, remainder is future work
- link back to Chapter 1: this is the control baseline, so extraction correctness is not negotiable

2.1.2 Hierarchical Structure of the Document

- four levels, plus threshold variants hanging off tasks
- the hierarchy is *implicit in the identifier* — 2.513 sits under process 2.510, but nothing on the page says so
- 3 synthetic chapter nodes were created to root the tree (say this — it is an honest modelling choice, not data)

Node type	Count
Chapter (synthetic)	3
Process	35
Task	138
Child task	102
Threshold variant	49
Total	327

Chapter	Nodes
1	23
2	212
3 (NSO)	92

- the chapter split is very uneven — chapter 2 carries 65% of all nodes. Worth one sentence: the document is not balanced, and any per-chapter statistic has to be read against that.

2.1.3 Table Layout and the Authority Notation

- two tables share each page, footnotes underneath both (your “collided tables”)
- small legend table, top right: reminds the reader what each *letter* means, and stops there — no levels
- main table: process / task / child task down the left, roles across the top as rotated headers, a code in each cell
- 5 base codes, 17 distinct notations in total — point back to the Chapter 1 table rather than repeating it

2.2.1 Global Overview and Node Distribution

- 327 nodes, 1,776 responsibility entries, 132 distinct roles
- average of roughly 6 responsibility entries per non-synthetic node
- reuse the node-type table above rather than a second one

2.2.2 Authority Code Distribution — your headline finding

- **47.7% of all 1,776 entries are “informed” (i).** Nearly half the document is not about who acts — it is about who must be told.
- that reframes what the DAM is: as much a communication map as an authority map. Nobody would guess that from reading a page; it only appears when you count.
- it also explains why FR4 matters — an answer that omits informed parties omits the single most common entry type in the document

Code	Count	Share
(i) informed	847	47.69%
I initiate	224	12.61%
C check/verify	203	11.43%
A approve	164	9.23%
R review	154	8.67%
R2	56	3.15%
R1	38	2.14%
A2	35	1.97%
A1	30	1.69%
A3	9	0.51%
C1	5	0.28%
C2	4	0.23%
I1, I2	2	0.11%
invalid: O, o, F, blank	5	0.28%

- **levels are mostly absent.** Bare C appears 203 times; C1 and C2 together only 9. So in practice the document records the *kind* of obligation far more often than its *degree*.
- **5 entries carry codes that are not in the legend at all** — “O”, “o”, “F”, and one blank. 0.28% of entries. Flag as a data-quality finding, carry it into 2.3.6.

CHECK BEFORE YOU WRITE: C3 and C4 do not appear in your data at all — zero occurrences. In Chapter 1 you present the C1/C2 vs C3/C4 split as a major trap. It *is* real, but it is a trap on the **question side**, not in the data: if a user asks “who must be consulted for X”, a naive system matching the word *consult* to the letter C would return the 203 bare-C check-and-verify entries, which is wrong. Your agent/authority.py prevents exactly that. Phrase it that way — as a query-interpretation hazard your design guards against — and it is bulletproof. Phrase it as “the document is full of this confusion” and the first examiner who counts will catch you.

2.2.3 Role and Actor Analysis

- 132 distinct roles; 10 entries (0.56%) never resolved to a header and are kept under an explicit unresolved placeholder rather than dropped
- strong concentration at the top, long tail beneath — many roles appear only once or twice
- **Country Manager / DDG appears 161 times**, making it the most connected actor in the document by a wide margin

Role	Entries
Country Manager / DDG	161
Sector Manager (HQ-based / Region-based)	94
Task Manager	88
RDG	88
Regional Implementation Support Manager (RISM)	85
C/R Program Officer	68
Sector Director	63
Concerned Sector VP	60
Procurement Officer	54
Regional Procurement Coordinator	51

- **the source document spells one role two different ways** — “Sector Manager” on page 43, “Sector Manger” on page 44. A typo in the source. Any system matching roles as strings sees two separate roles. Good, concrete example of why normalisation is a real step and not housekeeping.

2.2.4 Threshold Variants and Monetary Bands

- 49 threshold variants: same activity, different monetary band, different approver
- amounts are in UA (Units of Account) — define it on first use
- this is where the near-duplicate title problem lives
- **185 pairs of near-duplicate titles ($\geq 80\%$ similar, same opening words); 180 of them (97.3%) carry a different set of responsibilities; 124 have an outright different approver**

ID	Page	Title	Approver
2.432.4.a	43	Individual Consultant: $\leq$ UA 50,000	Sector Manager
2.433.4.c	44	Individual Consultant: $>$ UA 250,000	PRC

- titles 98.4% identical, facing pages, completely different approval route

Land: *The document's most dangerous feature is not complexity. It is similarity.*

2.2.5 Cross-References and Redirects

- 6 nodes carry an explicit redirect instead of responsibilities
- observed formats: “See DAM 16.100”, “Refer to Activities 2.114–2.117 in Section 2”
- some point *outside* the 79-page scope — 2.312.2 references 16.100/16.200/16.300/16.400, which do not exist in the extracted node set
- design consequence, and it is a good one: those edges are deliberately **not wired**, rather than pointing at nothing. A dangling edge would be worse than an absent one.

CHECK BEFORE YOU WRITE: 96 nodes have no responsibilities at all. 35 are processes, which is by design — `build_nodes.py` only extracts responsibilities for task / child_task / threshold_variant. But **42 tasks and 19 child tasks** also come out empty, and only 6 of the 96 have a redirect explaining why. Before you write this section, open the PDF at two or three of those and check: is the row genuinely blank in the source, or did the parser miss it? Either answer is fine and publishable — “genuinely blank” is a finding, “parser gap” is an honest limitation. Guessing is the only bad option.

2.2.6 Consolidated View

- one summary table pulling together: 327 nodes / 1,776 entries / 132 roles / 49 threshold variants / 6 redirects / 37 populated pages
- then one paragraph: what the exploration *changed*. Name at least two things — the informed-party share drove FR4, and the near-duplicate titles drove the confidence reporting on text search.

2.3.1 Text Layer Inspection

- the PDF has a real embedded text layer — it is not a scan, so no OCR was needed. Say this early; it rules out a whole category of problem a reader might assume you had.
- but the text layer is not trustworthy at the token level
- **the finding:** pdfplumber's own `extract_words()` is inconsistent. The identical visual pattern came out as one fused word “C13” in one cell, and as two separate words “C” + “13” in the next cell *on the same row*.
- consequence: text-pattern logic gives different, disagreeing answers depending on how the library happened to tokenise. Fused reads as level 1 / footnote 3; split reads as no level / footnote 13.

Land: *This is not a regex problem. It is a tokenisation problem sitting underneath the regex — which is why the fix had to move down to raw character coordinates.*

2.3.2 Rotated Column Headers

- role headers are rotated 90° on the page
- extraction returns them as free-floating strings with no binding to the column they label
- so the column-to-role mapping has to be rebuilt from x-positions — the page's meaning is positional, and the text layer does not carry position semantically
- ties directly back to Chapter 1's argument about why flattening destroys the answer

2.3.3 Fused Footnote Digits and Authority Codes — give this the most space

- this is the best piece of engineering in the thesis. Do not compress it.
- **the problem:** a cell shows a letter followed by one or more small digits, with no separator. Is a trailing digit the authority *level*, or a *footnote* marker? Both are superscript. Both are size 6 against the letter's size 9.
- **the rule you found:** compare each small character to the *letter's own top and bottom* — not to an estimated cell centre.
 - **bottom-aligned** → level digit (at most one)
 - **top-aligned** → footnote digit; consecutive top-aligned digits at the same height concatenate into one multi-digit footnote number
- **the measurements** — use them, they are what makes this credible:
 - 2.224.4, “A1” with footnote 21: level digit bottom 440.12 against the letter's 440.28; footnote digits top 430.64 against the letter's 431.28
 - 2.223.2 / 2.223.3, “C13”/“C14”: both trailing digits share identical top *and* bottom with each other — footnote alignment, not level alignment
- **the honest part, and write it in:** your first reading of “C13” was level 1 + footnote 3. The coordinates proved that wrong — there is no level digit in those cells at all. It is a bare C with a two-digit footnote 13. You corrected a wrong hypothesis with measurement. Say so; it is the difference between a rule you guessed and a rule you established.
- validated against real page screenshots, not just against itself
- 113 of 1,776 entries carry footnote references (6.36%); 19 distinct footnote numbers
- **known limitation:** one extracted footnote number reads 1617 — footnotes 16 and 17 fused. Carry it into 2.3.6 and

into Chapter 3’s limitations.

2.3.4 Cross-Row Text Bleeding and Title Corruption

- the “Task Manager1” case — a role stored as Task Manager1 & Project Team Members, no space before the 1
- scanned the whole dataset before concluding anything: **exactly 2 of 327 nodes** affected (2.224.1 and 2.325.2, same shared role text). Narrow, not systemic — and you only know that because you checked.
- traced to real character geometry, not guessed: the raw stream has “r” immediately followed by a shrunk “1” (size 3.24 against neighbours at 3.99–4.5) with no space character anywhere between them, while genuine spaces do exist around that digit elsewhere in the same run
- cause: `_strip_trailing_footnote_digit` only strips a shrunk digit at the *end* of a run. This one sits mid-run.
- **the story worth telling in full:** you attempted a fix at the header-merge boundary. Rebuilt, checked against the real node — **the text was completely unchanged**, proving the merge boundary was never where the bug lived. Worse, the full test suite caught a regression the “fix” introduced elsewhere: node 2.126’s “Sector Manager (HQ-based / Region-based)” became “HQ- based”. You reverted the change entirely and documented the case as a known limitation affecting 2 of 327 nodes.

Land: *Reverting a fix that did not fix its target and broke something else is the correct engineering decision, and examiners notice when a candidate makes it rather than shipping a plausible-looking patch.*

2.3.5 Split Rows in the Non-Sovereign Operations Chapter

- pull the detail from `docs/decisions.md` — search for the NSO / 3.110 / 3.225 / 3.226 entries
- structure it the same way as the others: what you saw, what you measured, what you changed, what remains

2.3.6 Consolidated Data Issues Summary

- one table: issue / scope / cause / status
- rows: tokenisation inconsistency (systemic, fixed via geometry) · rotated headers (systemic, fixed via x-position) · fused level/footnote digits (systemic, fixed via alignment rule) · fused footnote 1617 (1 case, open) · mid-run shrunk digit (2 of 327, open, deliberately) · invalid codes O/o/F/blank (5 of 1,776, open) · unresolved roles (10 entries, 0.56%, surfaced not dropped)
- close the chapter by pointing forward: every rule in Chapter 3 answers one row of this table

CHAPTER 3 — Pipeline and Knowledge Graph

3.1.1 Motivation for a Deterministic, Rule-Based Pipeline

- three reasons, in increasing order of strength:
 - **no labelled data.** 327 short titles, no query–title pairs. Not enough to train anything that would beat a general-purpose model, and “there isn’t enough data” is the honest answer if a jury asks why you didn’t.
 - **the founding premise.** The project exists to avoid a model deciding who approves a transaction. Letting a model do the extraction would undo the reason for building it.
 - **inspectability.** Every rule traces back to measured characters on a specific page. When a rule misfires you open that page and see why. A confidence score cannot be interrogated.

Land: *This is the same argument as Chapter 1’s case against commercial extraction services — say so explicitly, so the thesis reads as one argument rather than five chapters.*

3.1.2 Why Not a Distributed Processing Framework

- be direct: 79 pages, 327 nodes, 1,776 entries. The dataset is tiny.
- the hard part is per-character correctness, not volume — Spark or similar solves a problem this project does not have
- it would add real operational weight (cluster config, serialisation, debugging across workers) for zero benefit
- saying plainly that the data does not justify it is stronger than inventing a scalability story. An examiner asking “why no Spark?” is testing whether you know when *not* to use a tool.

3.1.3 Tooling

- give each tool its reason, not just its name — “pdfplumber, because the superscript rule needs per-character vertical offset and font size” beats “pdfplumber for PDF extraction”
- pdfplumber (character geometry) · Pydantic (fail at construction, not query time) · networkx (multigraph) · scikit-learn (TF-IDF) · pytest (374 tests / 23 files)

3.2.1 Global Architecture — 3.2.2 Data Flow

- four stages: offline extraction → validated cached node set → knowledge graph → agent and web layer
- **the decision worth naming:** extraction is offline and cached, so the running application never opens the PDF. It loads a prepared node set at startup.
- that was forced by a real deployment failure (Render out-of-memory) — and it turned out to be the right architecture anyway. Say both parts.
- module map: parsing/ · extraction/ · modeling/ · knowledge/ · agent/ · llm/ · webapp/ · schema/ — one line each on what it owns

3.3 Data Ingestion

- pdfplumber at character level, not word level
- justify it with the C13 finding from 2.3.1 — word-level tokenisation was demonstrably inconsistent, so the pipeline works one level below it
- this is a decision that looks like an implementation detail and is actually the project’s central technical choice

3.4.1–3.4.3 Row Reconstruction, Task Block Detection, Title and Reference Cleaning

- rows reassembled from character positions rather than taken from the library’s own table detection
- task blocks detected by identifier pattern, then attached to the enclosing process
- title cleaning had two real bugs worth writing up:
 - NOTE_PATTERN was stripping digits that appeared before hyphens, corrupting titles
 - several “See X” reference formats were silently corrupting titles *and* dead-ending real questions — two symptoms, one cause
- both are good short case studies: a cleaning rule that is slightly too greedy damages data quietly, and quiet damage is the worst kind

3.4.4 Geometric Attribution of Authority Codes

- restate the alignment rule briefly and point back to 2.3.3 — do not repeat the derivation
- **the validation, and the decision that came out of it:** geometry’s per-task action count was cross-checked against the already-trusted text-based count across all 79 pages. **228 tasks with actions; 197 matched; 31 did not (about 14%).** Mismatches ranged from off-by-one to badly off (2.412.1: text 3 / geometry 0; 3.642: text 1 / geometry 9).
- the spread indicated more than one remaining cause, not a single last bug
- **decision:** ship with the text-based count as the trusted source and geometry as best-effort supplementary detail, rather than keep chasing the 14%
- write that as a deliberate engineering judgment under a deadline, with the residual gap documented — not as something you failed to finish. Those are different claims, and the first one is both true and stronger.

3.4.5 Role Attribution

- column x-position maps a cell to the role whose rotated header sits above it
- anchor detection needed three rounds against real false positives, all on one title (2.223.2): raw character scans were picking up OCR-spaced letters — an “A” out of “A p p r a i s a l”
- fixes in sequence: require the whole word to match `^[ICRA]\d*$` (killed embedded letters like “PAR”), then add a word-adjacency check (killed isolated spaced single letters)
- 10 entries never resolved to a header (0.56%) and are kept under an explicit unresolved placeholder **rather than dropped**, so a “who is responsible” query still surfaces that something could not be attributed

Land: *Surfacing what you could not resolve, instead of silently dropping it, is the same principle as the agent refusing to answer rather than guessing.*

3.5 Node Construction and Validation

- Pydantic schema as the data contract — malformed records fail at construction, not at query time
- validation against real page screenshots throughout, not against the pipeline’s own output
- name at least one bug the validation caught that self-consistency would not have

3.6.1 Why a Graph

- the two founding questions from the original brief — “who approves task X” and “who must be informed for task X” — are *the same shape of question*: who is connected to X by a particular kind of edge, filtered by action
- a role such as “Country Manager / DDG” is a real recurring entity referenced from dozens of tasks. Worth being a first-class node with its own identity rather than a string repeated across 161 responsibility lists.
- **MultiDiGraph, not DiGraph** — and give the reason: a role can check a task *and later* approve it. That is two parallel edges between the same pair of nodes, not one edge carrying two labels. A plain DiGraph would silently collapse them.

3.6.2 Node and Edge Schema

- every extracted node becomes a graph node, keyed by its own DAM identifier
- one node per canonical role string, **prefixed** `role::` so a role name can never collide with a real DAM id — small decision, worth a sentence, it is the kind of thing that causes a silent disaster if skipped
- one shared `role::__unresolved__` node, kept visible
- three edge types: `contains` (parent→child) · `references` (task→task) · `responsible_for` (role→task, carrying action, level and footnote refs as edge attributes)
- `references` edges are wired **only if the target actually exists** in the node set — 2.312.2’s out-of-scope references are correctly skipped rather than pointed at nothing

3.6.3 Graph Statistics

- **459 nodes** (327 document nodes + 132 distinct roles including the unresolved placeholder) and **2,098 edges**
- **the validation to quote:** against the real screenshot of NSO 3.110 on page 58, querying task 3.111’s approvers returns exactly “Origination Sector Manager” (footnote 3) and “Supporting Dept. Division Manager” (footnote 4) — matching the page exactly, and not just the count of edges but the specific roles *and* their footnote numbers
- that is the strongest validation sentence in the chapter. Make sure it survives editing.

3.7.1 Execution and Caching

- the Render out-of-memory failure and what it forced: parse once at build time, ship the cached node set
- the running application loads prepared data, never the PDF
- consequence for reproducibility: the same input produces the same node set every time, and the deployed instance cannot drift from the tested one

3.7.2 Data Quality Outcomes

- 0.56% unresolved role rate (10 of 1,776)
- 374 tests across 23 files
- validation performed against real page screenshots at every stage, not against the pipeline’s own output

3.7.3 Observed Limitations — write this honestly, it is worth marks

- mid-run shrunk digit: 2 of 327 nodes, fix attempted, verified as ineffective, reverted deliberately
- fused footnote number 1617 (footnotes 16 and 17 run together): 1 case, open
- geometry vs text action-count mismatch: 31 of 228 tasks (14%), text treated as trusted source
- 5 entries carry codes absent from the legend (O, o, F, blank)
- 61 task and child-task nodes hold no responsibilities and no redirect — **check these against the PDF before you characterise them**
- scope: 79 of 250+ pages

Land: *A candidate who can list exactly where his system falls short reads as someone who understands it. Volunteering*

the limitation is much stronger than being asked for it.